CS235 Test 2 Answers

Question 1

Briefly discuss the following steps of transforming the ERD into relational schemas [6 marks]:

- i) Map binary relationships
- ii) Map associative entities
- iii) Map unary relationships

Answer:

i) Map Binary Relationships

Binary relationships in an Entity-Relationship Diagram (ERD) connect two entities. To transform these into relational schemas:

- **One-to-One (1:1)**: Include the primary key of one entity as a foreign key in the other entity's table. Choose the entity with higher participation to minimize null values.
- **One-to-Many (1:N)**: Add the primary key of the "one" side as a foreign key in the "many" side's table.
- **Many-to-Many (N:M)**: Create a new table containing the primary keys of both entities as foreign keys, forming a composite primary key. Include any relationship attributes in this table.

ii) Map Associative Entities

Associative entities represent relationships with attributes, typically from N:M relationships. To map them:

- Create a separate table for the associative entity.
- Include the primary keys of the related entities as foreign keys.
- The foreign keys, combined with any attributes of the associative entity, form the table, with the foreign keys often serving as the composite primary key.
- Ensure referential integrity by linking foreign keys to the respective entity tables' primary keys.

iii) Map Unary Relationships

Unary relationships involve an entity related to itself (e.g., an employee supervised by another employee). To map them:

- Add a foreign key in the entity's table that references its own primary key.

- For **1:1 or 1:N** relationships, the foreign key points to the primary key of the related instance (e.g., `ManagerID` referencing `EmployeeID`).
- For **N:M** relationships, create a new table with two columns, both foreign keys referencing the entity's primary key, forming a composite primary key.

Question 2

When is a relation said to be in the following normal forms [8 marks]:

- a) 1st Normal Form
- b) 2nd Normal Form
- c) 3rd Normal Form
- d) Boyce-Codd Normal Form

Answer:

a) First Normal Form (1NF)

A relation is in 1NF if:

- All attributes contain atomic (indivisible) values, with no repeating groups or arrays.
- Each attribute holds a single value from its domain.
- There are no multi-valued or composite attributes.

b) Second Normal Form (2NF)

A relation is in 2NF if:

- It is in 1NF.
- All non-key attributes are fully functionally dependent on the entire primary key, meaning no non-key attribute depends on a subset of a composite primary key (no partial dependencies).

c) Third Normal Form (3NF)

A relation is in 3NF if:

- It is in 2NF.
- There are no transitive dependencies, where a non-key attribute determines another non-key attribute. All non-key attributes must depend directly on the primary key.

d) Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if:

- It is in 3NF.
- For every functional dependency (X \to Y), (X) is a superkey. BCNF eliminates all non-trivial functional dependencies where the determinant is not a superkey, making it stricter than 3NF.

Question 3

The Functional Dependencies below are identified between various attributes of Parts in a company warehouse:

- FD1: PartNo → PartDescription
- FD2: PackSize, PartNo → Price
- FD3: PackSize, PartNo → FloorNo
- FD4: FloorNo → StorageLocation
  Produce the Relational Schema in the Second Normal Form and then in the Third Normal Form and Boyce-Codd Normal Forms [6 marks].

Answer:

Given the functional dependencies, we start with a relation: `Parts(PartNo, PartDescription, PackSize, Price, FloorNo, StorageLocation)`.

Step 1: Normalize to 2NF

The candidate key is `{PartNo, PackSize}` (from FD2 and FD3). Check for partial dependencies:

- FD1: `PartNo → PartDescription` is a partial dependency since `PartNo` is a subset of the candidate key.

Decompose to eliminate partial dependencies:

- **R1**: `Part(PartNo, PartDescription)` – Primary Key: `PartNo`
- **R2**: `PartPack(PartNo, PackSize, Price, FloorNo, StorageLocation)` – Primary Key: `{PartNo, PackSize}`

Step 2: Normalize to 3NF

For **R1**, no transitive dependencies exist, so it is in 3NF.
For **R2**, check for transitive dependencies:

- FD4: `FloorNo → StorageLocation` is a transitive dependency since `FloorNo` is not a key.

Decompose **R2**:

- **R2.1**: `Floor(FloorNo, StorageLocation)` – Primary Key: `FloorNo`
- **R2.2**: `PartPack(PartNo, PackSize, Price, FloorNo)` – Primary Key: `{PartNo, PackSize}`, Foreign Key: `PartNo` references `Part(PartNo)`, `FloorNo` references `Floor(FloorNo)`

Step 3: Check for BCNF

- **R1**: FD1 (`PartNo → PartDescription`) has `PartNo` as a superkey, so it is in BCNF.
- **R2.1**: FD4 (`FloorNo → StorageLocation`) has `FloorNo` as a superkey, so it is in BCNF.
- **R2.2**: FD2 (`PartNo, PackSize → Price`) and FD3 (`PartNo, PackSize → FloorNo`) have `{PartNo, PackSize}` as a superkey, so it is in BCNF.

Final Relational Schema

- **Part(PartNo, PartDescription)** – Primary Key: `PartNo`
- **PartPack(PartNo, PackSize, Price, FloorNo)** – Primary Key: `{PartNo, PackSize}`, Foreign Keys: `PartNo` references `Part(PartNo)`, `FloorNo` references `Floor(FloorNo)`
- **Floor(FloorNo, StorageLocation)** – Primary Key: `FloorNo`

Question 4

The questions below relate to the two relations shown:

- **Students(StudentNo, SName, Town, DOB, CourseNo, Fees)**
- **Enroll(StudentNo, CourseNo, DateEnrolled, Grade)**
  Write SQL queries to:
- a) Display all students enrolled in course number 01
- b) Display students with grade ‘A’ or ‘B’
- c) Display student details for students born after 01-01-88
- d) Display the number of students per course
- e) Display students who enrolled after 01-01-2016

Answer:

a) Display all students enrolled in course number 01

```
SELECT DISTINCT s.StudentNo, s.SName
FROM Students s
JOIN Enroll e ON s.StudentNo = e.StudentNo
```

```
WHERE e.CourseNo = '01';
```

Explanation: Joins `Students` and `Enroll` on `StudentNo`, filters for `CourseNo = '01'`, and uses `DISTINCT` to avoid duplicate student records.

b) Display students with grade ‘A’ or ‘B’

```
SELECT DISTINCT s.StudentNo, s.SName
FROM Students s
JOIN Enroll e ON s.StudentNo = e.StudentNo
WHERE e.Grade IN ('A', 'B');
```

Explanation: Joins `Students` and `Enroll`, filters for grades 'A' or 'B' using `IN`, and uses `DISTINCT` to list each student once.

c) Display student details for students born after 01-01-88

```
SELECT StudentNo, SName, Town, DOB, CourseNo, Fees
FROM Students
WHERE DOB > '1988-01-01';
```

Explanation: Selects all attributes from `Students` where `DOB` is after January 1, 1988.

d) Display the number of students per course

```
SELECT CourseNo, COUNT(DISTINCT StudentNo) AS StudentCount
FROM Enroll
GROUP BY CourseNo;
```

Explanation: Groups `Enroll` by `CourseNo` and counts distinct `StudentNo` values to determine the number of unique students per course.

e) Display students who enrolled after 01-01-2016

```
SELECT DISTINCT s.StudentNo, s.SName
FROM Students s
JOIN Enroll e ON s.StudentNo = e.StudentNo
WHERE e.DateEnrolled > '2016-01-01';
```

Explanation: Joins `Students` and `Enroll`, filters for enrollments after January 1, 2016, and selects distinct student details.